

Universidad del Valle de Guatemala

CC2008 - Introducción a la Programación Orientada a Objetos

Semestre II, 2026

Renta Movil

Josue David Garcia Mendez 26375

1 Propósito y alcance

Se propone una aplicación de consola en Java para registrar y consultar la flota de RentaMovil, cotizar alquileres, confirmar cobros, recibir devoluciones y consultar un reporte general. La solución organiza los datos comunes en una clase Vehiculo y las reglas particulares en Automovil, Motocicleta y CamionetaCarga.

La disponibilidad y los ingresos deben permanecer consistentes: únicamente un alquiler válido y confirmado ocupa el vehículo y aumenta el ingreso acumulado. Una cotización no reserva el vehículo; una devolución lo habilita de nuevo sin generar ni eliminar dinero.

Alcance de esta versión

El sistema administra automóviles, motocicletas y camionetas de carga durante una sola ejecución. Atiende a un empleado mediante un menú y conserva los datos en memoria. No requiere interfaz gráfica, archivos de datos ni base de datos. El caso excluye fechas, multas, reembolsos y reservaciones, y supone devoluciones dentro del plazo acordado.

Identificación del proyecto

Elemento	Nombre propuesto
Aplicación y proyecto de NetBeans	RentaMovil
Repositorio público de GitHub	rentamovil-herencia
Paquete de entrada	com.mycompany.rentamovil
Paquete del modelo y gestor	com.mycompany.rentamovil.modelo
Clase de inicio	RentaMovil

2 Requisitos funcionales

El actor principal es el empleado. La encargada utiliza las mismas consultas y el reporte; no se requiere crear cuentas ni implementar permisos de usuario.

ID	Operación y resultado requerido
RF01	Iniciar con al menos dos vehículos por categoría, placas diferentes, todos disponibles e ingresos de Q0.00.
RF02	Registrar un vehículo con sus datos comunes y específicos. Rechazar datos inválidos y placas repetidas; un registro válido queda disponible.
RF03	Consultar toda la flota y mostrar placa, marca, modelo, tarifa diaria, categoría, características particulares y disponibilidad.
RF04	Buscar un vehículo por placa. Si no existe, mostrar un mensaje claro y conservar todos los datos.

RF05	Cotizar por placa y cantidad de días. Mostrar características, disponibilidad y total; permitir la cotización aunque el vehículo esté alquilado.
RF06	Preparar un alquiler solo si el vehículo existe y está disponible. Calcular y mostrar el total antes de solicitar confirmación.
RF07	Confirmar el alquiler después de la aceptación. Cobrar el total una sola vez, sumarlo a los ingresos y marcar el vehículo como ocupado.
RF08	Cancelar antes de confirmar. Conservar la disponibilidad y el ingreso acumulado sin cambios.
RF09	Registrar una devolución únicamente si el vehículo existe y está alquilado. Dejarlo disponible sin cambiar los ingresos.
RF10	Generar un reporte con registrados, disponibles y alquilados por categoría, totales de la flota e ingresos acumulados.
RF11	Mantener el menú hasta elegir salir. Manejar entradas inválidas sin cierre inesperado y presentar montos con dos decimales.

Menú previsto

- 1 Registrar vehículo
- 2 Consultar flota
- 3 Cotizar alquiler
- 4 Realizar alquiler
- 5 Registrar devolución
- 6 Mostrar reporte general
- 0 Salir

La opción de alquiler incluye la consulta del precio y la pregunta de confirmación. No se necesita una opción separada para cancelar: responder que no a la confirmación devuelve el control al menú.

3 Información y reglas de cobro

Datos que se conservan durante la ejecución

Dato	Tipo Java	Uso
Placa	String	Identificador único del vehículo.
Marca y modelo	String	Descripción comercial; el modelo no se limita a un año numérico.
Tarifa diaria	BigDecimal	Monto base por cada día.
Disponibilidad	boolean	true: disponible; false: alquilado.
Cantidad de pasajeros	int	Solo para automóviles; entero positivo.
Transmisión automática	boolean	Solo para automóviles; determina el recargo diario.

Cilindraje	int	Solo para motocicletas; expresado en cc.
Capacidad en toneladas	BigDecimal	Solo para camionetas; admite decimales.
Flota	Map<String, Vehiculo>	Relaciona cada placa normalizada con un vehículo.
Ingresos acumulados	BigDecimal	Suma de los alquileres confirmados.

Los días solicitados, la respuesta de confirmación y la cotización son datos temporales de la operación. Los conteos del reporte se calculan a partir de la flota; no se guardan como contadores independientes.

Cálculo del total

Categoría y condición	Regla
Automóvil manual	Multiplicar la tarifa diaria por los días.
Automóvil automático	Sumar Q50.00 a la tarifa diaria y multiplicar por los días.
Motocicleta de hasta 250 cc	Multiplicar la tarifa diaria por los días. Exactamente 250 cc no tiene recargo.
Motocicleta de más de 250 cc	Multiplicar la tarifa diaria por los días y sumar Q75.00 una sola vez por alquiler.
Camioneta de carga	Sumar a la tarifa diaria Q100.00 por cada tonelada de capacidad máxima y multiplicar por los días.

El recargo de la camioneta depende de su capacidad máxima, no del peso transportado. Con tarifa de Q200.00, capacidad de 1.5 toneladas y 3 días, la base es Q600.00, el recargo Q450.00 y el total Q1,050.00.

Decisión de diseño: usar BigDecimal construido desde texto para importes y capacidad. Redondear únicamente el total final a dos decimales con HALF_UP y acumular exactamente ese importe cobrado. El enunciado exige dos decimales, pero no fija un método de redondeo; esta política lo concreta.

4 Validaciones y conservación del estado

Entrada u operación	Condición de validez
Placa	No nula ni vacía después de quitar espacios externos. Se normaliza a mayúsculas antes de almacenar y buscar. La unicidad abarca toda la flota.
Marca y modelo	Se propone exigir texto no vacío para evitar registros incompletos. Es una validación adicional al mínimo explícito del enunciado.
Tarifa diaria	BigDecimal estrictamente mayor que cero.
Pasajeros y cilindraje	Enteros estrictamente mayores que cero.
Capacidad de carga	BigDecimal estrictamente mayor que cero; por ejemplo, 1.5 toneladas.

Días	int positivo. Rechazar fracciones, cero, negativos, texto y números fuera del rango de int.
Confirmación y transmisión	Aceptar únicamente las opciones anunciadas. Volver a pedir el dato ante una respuesta diferente.
Alquiler	Placa existente, días válidos, vehículo disponible y aceptación explícita del total.
Devolución	Placa existente y vehículo actualmente alquilado.
Formato de números	Leer líneas completas y convertir dentro de un manejo de errores. Para decimales se indicará el punto como separador y no se admitirán separadores de miles.

Invariantes del sistema

Cada placa identifica un único vehículo y permanece inmutable después del registro. Todo vehículo tiene exactamente uno de dos estados: disponible o alquilado. No puede confirmarse otro alquiler mientras permanezca ocupado.

Los ingresos empiezan en Q0.00 y aumentan solo por confirmaciones exitosas. Registrar, consultar, cotizar, cancelar, devolver y rechazar una operación no cambian el ingreso acumulado. Una devolución tampoco borra cobros anteriores.

En cada categoría, registrados equivale a disponibles más alquilados. La suma de las categorías coincide con el total de la flota. El reporte consulta el estado actual sin modificarlo.

Tratamiento de errores

El modelo y el gestor rechazan argumentos inválidos con `IllegalArgumentException` y estados incompatibles con `IllegalStateException`. `RentaMovil` captura estos errores esperados, informa la causa y permite continuar. La conversión numérica captura `NumberFormatException`; no se utiliza un catch genérico para ocultar fallas de programación.

Toda validación debe completarse antes de modificar la flota, la disponibilidad o los ingresos. La aplicación procesa una operación de consola a la vez; no se plantea acceso concurrente.

5 Clases y organización de responsabilidades

Clase	Responsabilidad
Vehiculo — abstracta	Conservar y validar los datos comunes y la disponibilidad. Calcular el costo base, incorporar el recargo correspondiente y redondear el total. Delegar los detalles específicos a las subclases.
Automovil	Conservar la cantidad de pasajeros y el tipo de transmisión. Aplicar el recargo diario cuando la transmisión es automática.
Motocicleta	Conservar el cilindraje. Aplicar el cargo adicional único cuando supera los 250 cc.

CamionetaCarga	Conservar la capacidad máxima en toneladas. Calcular el recargo según esa capacidad y los días de alquiler.
GestorRentaMovil	Administrar la flota, impedir placas repetidas, coordinar cotizaciones, alquileres y devoluciones, acumular los ingresos y generar el reporte general.
RentaMovil	Contener el método main, gestionar el menú, leer y validar el formato de las entradas, presentar los resultados y solicitar la confirmación del alquiler.

Relaciones previstas

Automovil, Motocicleta y CamionetaCarga extienden directamente a Vehiculo, porque cada una representa una categoría de vehículo. Las tres son subclases de una misma clase base y no heredan unas de otras.

GestorRentaMovil mantiene una colección de cero o más objetos de tipo Vehiculo, que pueden pertenecer a cualquiera de sus subclases. Esta relación se representa mediante una asociación unidireccional con multiplicidad 0..* en el extremo de Vehiculo.

Se utiliza una asociación porque el gestor necesita mantener referencias a los vehículos registrados. El diseño no impone una dependencia de ciclo de vida ni una propiedad exclusiva que deba representarse mediante composición.

RentaMovil mantiene una referencia a un objeto GestorRentaMovil. También depende de las clases concretas para construir los vehículos con los datos introducidos por el usuario o con los datos iniciales de demostración. Utiliza referencias de tipo Vehiculo para consultar las descripciones y la disponibilidad.

Paquetes y protección del estado

La clase RentaMovil pertenece al paquete `com.mycompany.rentamovil`. Las clases Vehiculo, Automovil, Motocicleta, CamionetaCarga y GestorRentaMovil pertenecen al paquete `com.mycompany.rentamovil.modelo`.

Los métodos `ocupar()` y `liberar()` de Vehiculo tienen acceso de paquete. Por ello, pueden ser utilizados por las clases del paquete del modelo, pero no directamente por RentaMovil, que se encuentra en otro paquete. Por responsabilidad de diseño, el gestor coordina estas transiciones y las invoca después de validar la operación.

Todos los atributos son privados. Los datos descriptivos y particulares de los vehículos son final y reciben sus valores en los constructores. No se exponen setters generales para modificar la placa, la tarifa, la disponibilidad o los ingresos acumulados.

El mapa que contiene la flota permanece privado. Su referencia es final, lo que impide sustituirla por otro mapa, pero permite incorporar nuevos vehículos a la colección existente.

El método `getVehiculos()` devuelve una copia no modificable de la lista de referencias a los vehículos. Quien recibe esa lista no puede agregar, eliminar ni reemplazar elementos en ella. Sin embargo, los elementos siguen siendo los mismos objetos administrados por el gestor; no se crean copias independientes de cada vehículo.

Convenciones de las fichas

Se utiliza la notación `+` para public, `-` para private, `#` para protected y `~` para acceso de paquete.

En los atributos y parámetros, el tipo aparece después de dos puntos. En los métodos, el tipo indicado después de los paréntesis corresponde al retorno; void significa que no se devuelve un valor. Los constructores no tienen tipo de retorno.

Los calificadores abstract, final y static se indican cuando corresponden. En el diagrama UML, los elementos abstractos se representan en cursiva y los elementos estáticos se subrayan.

No se crean clases Cliente o Alquiler en esta versión, porque el caso no requiere datos personales, historial individual de operaciones ni fechas. La disponibilidad de cada vehículo y el ingreso acumulado permiten resolver las operaciones solicitadas. Si posteriormente se requiere trazabilidad por transacción, será necesario ampliar el modelo.

6 Clase Vehículo

Clase abstracta del paquete modelo. Su constructor recibe los datos comunes, los valida y establece disponible en true. No puede crearse directamente con new Vehículo; cada subclase llama a este constructor mediante super.

Atributo	Tipo y visibilidad	Propósito
placa	private final String	Identificador normalizado e inmutable.
marca	private final String	Marca sin espacios externos.
modelo	private final String	Modelo sin espacios externos.
tarifaDiaria	private final BigDecimal	Tarifa base positiva.
disponible	private boolean	Estado inicial true; cambia solo al alquilar o devolver.

Constructor y métodos

Firma propuesta	Propósito y resultado
# Vehículo(placa: String, marca: String, modelo: String, tarifaDiaria: BigDecimal)	Valida e inicializa los datos comunes. Rechaza nulos, placa vacía, marca/modelo vacíos y tarifa no positiva.
~ static normalizarPlaca(placa: String): String	Valida, aplica trim y convierte a mayúsculas con Locale.ROOT. Conserva los caracteres internos; no inventa un patrón legal de placa.
+ getPlaca(): String	Devuelve el identificador normalizado para búsquedas y registro.
+ isDisponible(): boolean	Devuelve el estado actual sin modificarlo.
+ abstract getCategoria(): String	Cada subclase devuelve un nombre estable de categoría para agrupar el reporte.
+ final getDescripcion(): String	Reúne placa, marca, modelo, tarifa, disponibilidad, categoría y detalle específico en un texto.

# abstract getDetalleEspecifico(): String	Devuelve los datos propios de la categoría, sin repetir los comunes.
+ final calcularCosto(dias: int): BigDecimal	Valida días positivos, multiplica tarifa por días, suma calcularRecargo(dias) y redondea el total a dos decimales. No cambia estado ni ingresos.
# abstract calcularRecargo(dias: int): BigDecimal	Devuelve el recargo definido por la subclase. Recibe días ya validados por calcularCosto.
~ ocupar(): void	Exige que esté disponible y cambia disponible a false; rechaza un segundo alquiler.
~ liberar(): void	Exige que esté alquilado y cambia disponible a true; rechaza una devolución improcedente.

Los dos métodos de transición tienen acceso de paquete. La interfaz de consola no puede llamarlos directamente. El gestor debe ser el único colaborador que los invoque dentro del diseño propuesto.

7 Clases especializadas

Las tres clases son públicas y extienden Vehiculo. Cada instancia contiene los datos comunes encapsulados en Vehiculo. Las subclases heredan el cálculo general, agregan sus atributos particulares e implementan los métodos abstractos correspondientes.

Atributos particulares

Clase	Atributo	Tipo y visibilidad	Propósito
Automovil	cantidadPasajeros	private final int	Cantidad positiva de pasajeros.
Automovil	automatica	private final boolean	true para transmisión automática.
Motocicleta	cilindraje	private final int	Cilindraje positivo en cc.
CamionetaCarga	capacidadToneladas	private final BigDecimal	Capacidad máxima positiva con decimales.

Constructores e inicialización

+ Automovil(placa: String, marca: String, modelo: String, tarifaDiaria: BigDecimal, cantidadPasajeros: int, automatica: boolean)

Valida cantidadPasajeros > 0 y conserva la transmisión recibida. RentaMovil valida la respuesta antes de convertirla a boolean.

+ Motocicleta(placa: String, marca: String, modelo: String, tarifaDiaria: BigDecimal, cilindraje: int)

Valida cilindraje > 0 y guarda el valor recibido. La condición de recargo es mayor que 250, sin incluir 250.

+ CamionetaCarga(placa: String, marca: String, modelo: String, tarifaDiaria: BigDecimal, capacidadToneladas: BigDecimal)

Rechaza capacidad nula o no positiva y conserva la capacidad máxima recibida, incluidos sus decimales.

Métodos que redefine cada subclase

Firma	Automovil	Motocicleta	CamionetaCarga
+ getCategoria(): String	"Automóvil"	"Motocicleta"	"Camioneta de carga"
# getDetalleEspecifico(): String	Pasajeros y transmisión.	Cilindraje con la unidad cc.	Capacidad máxima en toneladas.
# calcularRecargo(dias: int): BigDecimal	Q50.00 por día si es automático; cero si es manual.	Q75.00 si supera 250 cc; cero en otro caso. Se cobra una sola vez.	Q100.00 por capacidadToneladas por días.

Las redefiniciones usan @Override. No cambian la disponibilidad ni los ingresos. calcularCosto, heredado de Vehiculo, suma el recargo al costo base y aplica el redondeo final; las subclases no duplican ese procedimiento.

8 Clase GestorRentaMovil

Clase pública del paquete modelo. Centraliza las operaciones que afectan a la flota y al ingreso acumulado. No lee del teclado ni imprime; devuelve resultados para que RentaMovil los presente.

Atributo	Tipo y visibilidad	Inicialización y propósito
vehiculos	private final Map<String, Vehiculo>	new LinkedHashMap<>(); conserva el orden de registro y permite localizar por placa.
ingresosAcumulados	private BigDecimal	Cero con dos decimales; conserva los cobros confirmados.
Firma propuesta		Propósito y resultado
+ GestorRentaMovil()		Crea la colección vacía y establece ingresos en Q0.00. RentaMovil agrega después los seis vehículos de demostración.
+ registrarVehiculo(vehiculo: Vehiculo): void		Rechaza null, placa duplicada o vehículo que no esté disponible. Agrega el objeto únicamente después de validar.
+ buscarVehiculo(placa: String): Vehiculo		Normaliza la placa y devuelve el vehículo. Si no existe, lanza IllegalArgumentException con un mensaje claro.
+ getVehiculos(): List<Vehiculo>		Devuelve una copia no modificable de los valores del mapa, conservando el orden. No expone el mapa interno.
+ cotizar(placa: String, dias: int): BigDecimal		Busca el vehículo y llama a calcularCosto. Permite vehículos ocupados y no modifica ningún dato.

+ confirmarAlquiler(placa: String, dias: int): BigDecimal	Se invoca solo tras la aceptación. Revalida existencia, días y disponibilidad, calcula el total, ocupa el vehículo y suma el cobro una vez. Devuelve el importe cobrado.
+ devolverVehiculo(placa: String): void	Busca el vehículo y llama a liberar. Conserva íntegros los ingresos.
+ getIngresosAcumulados(): BigDecimal	Devuelve el ingreso acumulado para consultas y verificación.
+ generarReporte(): String	Agrupar por getCategory(), cuenta registrados, disponibles y alquilados, agrega totales generales e ingresos y devuelve un resumen de texto.

Orden de la confirmación

Antes de cualquier cambio se calcula el total y el nuevo ingreso en variables locales. Después se llama a ocupar y se asigna el nuevo ingreso. Así, un error de validación o de disponibilidad deja los datos anteriores intactos. El importe mostrado y el confirmado utilizan el mismo cálculo; las tarifas son inmutables y el flujo de consola es secuencial.

La cancelación no necesita un método del gestor: RentaMovil no invoca confirmarAlquiler cuando la respuesta es negativa. No existe una operación de deshacer un alquiler ya cobrado, porque el alcance excluye reembolsos.

El reporte deriva las categorías de los vehículos registrados y no guarda contadores. Como decisión de presentación para esta aplicación pequeña, el gestor devuelve el resumen textual; RentaMovil se limita a imprimirlo.

9 Clase RentaMovil y manejo de la consola

Clase pública del paquete com.mycompany.rentamovil. Sus métodos de instancia son privados; el punto de entrada estático crea RentaMovil e inicia el menú. No contiene fórmulas de alquiler ni modifica directamente la disponibilidad.

Atributo	Tipo y visibilidad	Propósito
entrada	private final Scanner	Lector único de System.in.
gestor	private final GestorRentaMovil	Coordinador de las operaciones del negocio.

Firma propuesta	Propósito y resultado
- RentaMovil ()	Crea Scanner y GestorRentaMovil, y llama una sola vez a cargarDatosIniciales.
+ static main(args: String[]): void	Punto de entrada de Java. Los argumentos de línea de comandos no se utilizan.

- cargarDatosIniciales(): void	Construye y registra los seis vehículos indicados en la sección 11.
- ejecutar(): void	Repite el menú, atiende la opción y captura errores esperados hasta salir.
- mostrarMenu(): void	Muestra las opciones de la sección 2.
- registrarVehiculo(): void	Pide categoría, datos comunes y datos específicos; construye la subclase y solicita su registro al gestor.
- mostrarFlota(): void	Recorre getVehiculos() y muestra getDescripcion() de cada objeto.
- cotizarAlquiler(): void	Pide placa y días, muestra getDescripcion() y el costo. Si está ocupado, indica que puede cotizarse pero no alquilarse.
- realizarAlquiler(): void	Comprueba disponibilidad, muestra características y total, solicita aceptación y, únicamente si se acepta, llama a confirmarAlquiler.
- registrarDevolucion(): void	Pide la placa, solicita la devolución al gestor y muestra el resultado.
- mostrarReporte(): void	Imprime el texto que devuelve generarReporte().
- leerTexto(mensaje: String): String	Muestra el mensaje y obtiene texto no vacío sin espacios externos.
- leerEntero(mensaje: String, minimo: int, maximo: int): int	Repite la lectura hasta obtener un entero en el intervalo inclusivo. Para días, pasajeros y cc usa 1 e Integer.MAX_VALUE.
- leerDecimalPositivo(mensaje: String): BigDecimal	Repite la lectura hasta obtener un decimal positivo con el formato anunciado.
- leerSiNo(mensaje: String): boolean	Acepta S o N sin distinguir mayúsculas; devuelve true o false y vuelve a pedir otras entradas.

Las lecturas usan nextLine y convierten después el texto. Esto evita mezclar lecturas numéricas con saltos de línea pendientes. La opción 0 finaliza normalmente; el cierre de la entrada también se puede tratar como una salida controlada.

10 Colaboración y justificación de la herencia

Cotización y confirmación de un alquiler

1. RentaMovil solicita la placa y la cantidad de días, y valida el formato de las entradas. El gestor normaliza la placa y localiza el vehículo. Cuando el modelo o el gestor detectan argumentos inválidos o un estado incompatible con la operación, lanzan la excepción correspondiente.

RentaMovil captura los errores esperados y muestra un mensaje claro, sin modificar los datos del sistema.

2. Para una cotización, el gestor llama a `calcularCosto(dias)`. Vehiculo valida que los días sean positivos, calcula el costo base, ejecuta el método `calcularRecargo(dias)` correspondiente al tipo real del objeto y redondea el total. RentaMovil muestra la descripción, la disponibilidad y el precio, aunque el vehículo esté ocupado. Esta consulta no modifica la disponibilidad ni los ingresos.
3. Para realizar un alquiler, RentaMovil consulta primero la disponibilidad del vehículo localizado por el gestor. Si está ocupado, informa que no puede alquilarse. Si está disponible, obtiene y muestra el total antes de solicitar la confirmación. Si la respuesta es N, termina la operación sin invocar `confirmarAlquiler()` y conserva el estado anterior.
4. Si la respuesta es S, RentaMovil invoca `confirmarAlquiler(placa, dias)`. El gestor vuelve a validar la existencia del vehículo, los días y la disponibilidad. Antes de modificar el estado, calcula el total y el nuevo ingreso acumulado en variables locales. Después ocupa el vehículo y actualiza los ingresos con el monto completo del alquiler. Una nueva solicitud de alquiler sobre ese vehículo se rechaza mientras continúe ocupado.
5. Para una devolución, RentaMovil solicita la placa y la entrega al gestor. El gestor localiza el vehículo y llama a `liberar()`. Este método comprueba que esté alquilado antes de cambiar su disponibilidad. Si ya está disponible, lanza `IllegalStateException` sin modificar el estado. Los ingresos permanecen sin cambios.

Elementos comunes y elementos variables

La herencia permite definir una sola vez los datos comunes de los vehículos: placa, marca, modelo, tarifa diaria y disponibilidad. Estos atributos privados permanecen encapsulados en Vehiculo; las subclases no los declaran nuevamente ni acceden directamente a ellos.

Cada objeto de una subclase contiene el estado definido por la clase base. El constructor de la subclase invoca mediante `super(...)` el constructor de Vehiculo, que valida e inicializa los datos comunes. Después se validan e inicializan los atributos particulares: pasajeros y transmisión en Automovil, cilindraje en Motocicleta y capacidad máxima en CamionetaCarga.

El método final `calcularCosto(dias)` concentra la validación de días, el cálculo de la tarifa base, la incorporación del recargo y el redondeo del total. El método protegido y abstracto `calcularRecargo(dias)` se implementa en cada subclase según su regla de cobro.

Este uso de polimorfismo permite que el gestor trabaje con referencias de tipo Vehiculo y solicite el costo sin seleccionar manualmente una fórmula según la categoría. La implementación de `calcularRecargo(dias)` se determina por el tipo real del objeto.

El método final `getDescripcion()` reúne los datos comunes y utiliza `getCategoria()` y `getDetalleEspecifico()` para incorporar la información particular. Así, las subclases proporcionan sus características sin duplicar la construcción de la descripción general.

Vehiculo es abstracta porque esta versión registra vehículos de categorías concretas. Sus métodos abstractos exigen que cada clase concreta proporcione su categoría, sus detalles y su recargo. Los atributos privados protegen el estado, mientras que los métodos protegidos permiten implementar el comportamiento específico de cada subclase.

Incorporación de una nueva categoría

Si una nueva categoría conserva el esquema de tarifa diaria más un recargo, se agrega una subclase de Vehículo con sus atributos y constructor. Esta clase debe implementar getCategory(), getDetalleEspecifico() y calcularRecargo(dias).

También se amplía el menú y la captura de datos de RentaMovil para permitir la creación de la nueva categoría. Se actualizan los datos de demostración, las pruebas, el diagrama UML y el análisis. Las operaciones generales del gestor, como registro, búsqueda, cotización, confirmación y devolución, pueden conservarse.

El reporte obtiene las categorías mediante getCategory() y agrupa los vehículos registrados. Por ello, no necesita una condición específica para cada categoría. El menú de registro sí conoce las clases concretas, porque debe solicitar sus datos particulares e invocar sus constructores.

Si una categoría futura utiliza una política de cobro incompatible con el cálculo previsto, o necesita datos adicionales como kilometraje, será necesario revisar la abstracción de cálculo y los datos de la operación. La propuesta facilita incorporar categorías compatibles con el modelo actual, pero no supone que cualquier regla futura pueda añadirse sin modificarlo.

11 Datos iniciales y cálculos esperados

RentaMovil carga una sola vez los siguientes datos propuestos. Todas las placas son distintas, todos los vehículos empiezan disponibles y el gestor conserva ingresos de Q0.00. Los nombres y valores se eligen para demostrar las reglas del ejercicio.

Placa	Vehículo	Tarifa diaria	Datos específicos
P001AAA	Toyota Corolla Automóvil	Q150.00	5 pasajeros; manual
P002AAA	Honda Civic Automóvil	Q180.00	5 pasajeros; automático
M001AAA	Yamaha FZ25 Motocicleta	Q80.00	250 cc
M002AAA	Kawasaki Ninja Motocicleta	Q120.00	400 cc
C001AAA	Hyundai H100 Camioneta	Q200.00	1.5 toneladas
C002AAA	Isuzu NPR Camioneta	Q250.00	2.0 toneladas

Pruebas de cálculo planificadas

Las cotizaciones siguientes se ejecutarán sin confirmar alquileres. En todos los casos, los ingresos deben seguir en Q0.00 y los seis vehículos deben permanecer disponibles. Los resultados obtenidos se completarán al ejecutar el programa.

ID	Entrada	Resultado esperado
----	---------	--------------------

PC01	P001AAA por 3 días	Base Q450.00, recargo Q0.00; total Q450.00.
PC02	P002AAA por 3 días	Base Q540.00, recargo Q150.00; total Q690.00.
PC03	M001AAA por 3 días	250 cc sin recargo; total Q240.00.
PC04	M002AAA por 1 día	Q120.00 más Q75.00; total Q195.00.
PC05	M002AAA por 3 días	Q360.00 más Q75.00 una sola vez; total Q435.00.
PC06	C001AAA por 3 días	Q600.00 más Q450.00; total Q1,050.00.
PC07	C002AAA por 2 días	Q500.00 más Q400.00; total Q900.00.

Cobertura adicional de límites

Se probarán motocicletas temporales de 249, 250 y 251 cc con tarifa Q80.00 y 2 días. Los totales esperados son Q160.00, Q160.00 y Q235.00, respectivamente. Cada ejecución utilizará una placa nueva o una instancia de prueba aislada.

El reporte al iniciar debe indicar, para cada categoría, 2 registrados, 2 disponibles y 0 alquilados. Los totales generales deben ser 6 registrados, 6 disponibles, 0 alquilados e ingresos de Q0.00.

12 Plan de pruebas de estado y validación

La secuencia PE01 a PE09 parte de una ejecución nueva y se realiza en el orden indicado. Cada resultado obtenido está pendiente de ejecución; después se añadirá una transcripción o captura breve como evidencia.

ID	Acción	Resultado esperado
PE01	Cotizar P002AAA por 3 días.	Q690.00; disponible; ingresos Q0.00.
PE02	Solicitar ese alquiler y cancelar.	Sigue disponible; ingresos Q0.00.
PE03	Confirmar P002AAA por 3 días.	Queda alquilado; ingresos Q690.00.
PE04	Intentar alquilarlo de nuevo.	Rechazo; sigue alquilado; ingresos Q690.00.
PE05	Cotizarlo ocupado por 1 día.	Q230.00; sigue alquilado; ingresos Q690.00.
PE06	Consultar el reporte.	Automóviles: 2 registrados, 1 disponible y 1 alquilado. Global: 6, 5 y 1; ingresos Q690.00.
PE07	Devolver P002AAA.	Disponible; ingresos Q690.00.
PE08	Intentar devolverlo otra vez.	Rechazo; disponible; ingresos Q690.00.
PE09	Alquilarlo nuevamente por 1 día.	Alquilado; ingresos Q920.00.

Operaciones rechazadas y recuperación

ID	Entrada o situación	Resultado esperado
PV01	Registrar una placa vacía o repetir P001AAA.	Rechazo; no se agrega ningún vehículo.

PV02	Registrar " p001aaa " en otra categoría.	Se detecta duplicado después de normalizar.
PV03	Tarifa, pasajeros, cc o capacidad en cero o negativos; pasajeros/cc fraccionarios.	Rechazo; flota e ingresos se conservan.
PV04	Días 0, -1, 2.5, texto o fuera del rango de int.	Mensaje de validación y nueva lectura; sin alquiler.
PV05	Placa inexistente al cotizar, alquilar o devolver.	Mensaje claro; estado e ingresos sin cambios.
PV06	Letras en un campo numérico, opción fuera del menú o confirmación distinta de S/N.	Se vuelve a pedir el dato; el programa continúa.
PV07	Registro válido adicional de una categoría.	Aumenta el total de esa categoría, queda disponible y no cambia el ingreso.
PV08	Salir y abrir una ejecución nueva.	Se restablecen los seis vehículos iniciales disponibles e ingresos Q0.00.

Prueba adicional:

File Edit View Navigate Source Refactor Run Debug Profile Team Tools Window Help

RentaMovil - Apache NetBeans IDE 31

Search (Ctrl+I)

Projects Files Favorites Services

AgenciaDetectives [main]

foodsmanger 2026 [main]

RentaMovil [main]

Source Packages

com.mycompany.rentamovil

RentaMovil.java [-/M]

com.mycompany.rentamovil.modelo

Automovil.java

CamionetaCarga.java

GestorRentaMovil.java

Motocicleta.java

Vehiculo.java [-/M]

Test Packages

Dependencies

Java Dependencies

Project Files

Source History

```
29 aplicacion.ejecutar();
30
31
32 private void cargarDatosIniciales() {
33     gestor.registrarVehiculo(new Automovil(
34         "P001AAA", "Toyota", "Corolla",
35         new BigDecimal("150.00"), 5, false));
36
37     gestor.registrarVehiculo(new Automovil(
38         "P002AAA", "Honda", "Civic",
39         new BigDecimal("180.00"), 5, true));
40
41     gestor.registrarVehiculo(new Motocicleta(
42         "M001AAA", "Yamaha", "FZ25",
43         new BigDecimal("80.00"), 250));
44 }
```

Output - Run (RentaMovil)

```
>> Seleccione una opcion: 1
>>
REGISTRAR VEHICULO
1. Automovil
2. Motocicleta
3. Camioneta de carga
Seleccione la categoria: 1
Placa: 600FE
Marca: toyot
Modelo: corolla
Tarifa diaria en Q (ejemplo: 150.90): 600
Cantidad de pasajeros: 4
¿Transmision automatica? (S/N): s
Vehiculo registrado correctamente: 600FE

RENTA MOVIL
1. Registrar vehiculo
2. Consultar flota
3. Cotizar alquiler
4. Realizar alquiler
```

Acción	Resultado esperado	Resultado obtenido
Intentar registrar nuevamente P001AAA	Rechazar la placa repetida.	Se mostró: “Ya existe un vehiculo con la placa P001AAA”.

Registrar 600PE	Aceptar un vehículo con placa nueva.	Se mostró el registro exitoso de 600PE. Los reportes posteriores contienen 7 vehículos.
Cotizar 600PE por 8 días	Q5200.00; vehículo disponible.	Total cotizado Q5200.00 y estado Disponible.
Confirmar ese alquiler	Registrar un cobro de Q5200.00.	Se mostró: “Alquiler confirmado. Cobro registrado: Q5200.00”.
Devolverlo y consultar el reporte	Vehículo disponible; conservar Q5200.00 de ingresos.	Reporte: 7 disponibles, 0 alquilados e ingresos Q5200.00.
Alquilarlo nuevamente y consultar el reporte	Acumular Q10400.00; tener un vehículo alquilado.	Reporte: 6 disponibles, 1 alquilado e ingresos Q10400.00.

Alquiler realizado

```

Categoria: Automovil
Placa: P002AAA
Marca: Honda
Modelo: Civic
Tarifa diaria: Q180.00
Estado: Disponible
Cantidad de pasajeros: 5
Transmision: Automatica
Total por cobrar: Q230.00
♦ Confirma el alquiler? (S/N): s
Alquiler confirmado. Cobro registrado: Q230.00

RENTA MOVIL
1. Registrar vehiculo

```

Reporte:

REPORTE DE FLOTA

Automovil

Registrados: 2 | Disponibles: 1 | Alquilados: 1

Motocicleta

Registrados: 2 | Disponibles: 2 | Alquilados: 0

Camioneta de carga

Registrados: 2 | Disponibles: 2 | Alquilados: 0

TOTALES DE LA FLOTA

Registrados: 6 | Disponibles: 5 | Alquilados: 1

Ingresos acumulados: Q690.00

Cancelación:

Categoria: Automovil

Placa: P002AAA

Marca: Honda

Modelo: Civic

Tarifa diaria: Q180.00

Estado: Disponible

Cantidad de pasajeros: 5

Transmision: Automatica

Total por cobrar: Q690.00

◆ Confirma el alquiler? (S/N): n

Alquiler cancelado. No se realizo ningun cobro.

Rent (Rentamovil)

REPORTE DE FLOTA

Automovil

Registrados: 2 | Disponibles: 2 | Alquilados: 0

Motocicleta

Registrados: 2 | Disponibles: 2 | Alquilados: 0

Camioneta de carga

Registrados: 2 | Disponibles: 2 | Alquilados: 0

TOTALES DE LA FLOTA

Registrados: 6 | Disponibles: 6 | Alquilados: 0

Ingresos acumulados: Q0.00

Diagrama UML:

